

实验报告（Lab0）

刘嘉媛 25300680178

1. 文档问题回答

（1）还没有多人协同开发的经历。

（2）暂存区中储存的是预期下一次提交的内容，提交时只将暂存区的内容存入 `git` 仓库。

在提交过程中新增一个过程，可以避免误触提交，也方便再次检验筛查，保证提交内容的正确性。同时，暂存区的存在方便实验的阶段化，暂存区一组的修改都相对独立，逻辑清晰，回溯时方便定位，而一次性提交则保证实验的整体性，使其更加连贯。

（3）“`git branch`”仅列出本地分支，并在当前分支前标记“*”。

“`git branch -a`”列出所有分支，包括本地分支和远程跟踪分支。

2. ~

3. 《Gitflow 使用规范》

`Gitflow` 是一套基于 `Git` 分支的协作管理规范，核心目标是让不同分支各司其职，避免多人协作时的代码冲突和版本混乱，各公司的分支规范虽各有差异，但万变不离其宗都有 `Gitflow` 的影子。

它定义了五类关键分支：`master` 为永久分支，对应线上生产环境，存放所有已发布版本，每次提交后立即打 `tag`；

`develop` 也是永久分支，从 `master` 分离，作为开发主线和团队公共分支；

`feature` 为临时分支，从 `develop` 创建，一个功能对应一个分支，完成后合并回 `develop` 并删除；

`release` 同为临时分支，从 `develop` 创建，用于测试阶段修 `bug` 和更新版本号，完成后合并到 `master` 和 `develop` 并删除；

`hotfix` 则从 `master` 创建，用于紧急修复线上 `bug`，完成后同样合并到 `master` 和 `develop` 并删除。

贯穿其中的核心原则是“从哪里来，最后回到哪里去”。实际使用时既可以手动执行 `checkout`、`merge --no-ff`、打 `tag` 等原生命令，也可以借助 `git-flow` 扩展库用封装命令自动完成合并与 `tag` 创建。此外还约定版本号采用“主版本.次版本.修订号”格式，`master` 上打 `tag` 不带 `v`，分支命名官方推荐使用连字符。

《语义化版本》

语义化版本控制（`SemVer`）是为解决软件开发中“依赖地狱”问题而提出的一套版本号规范。在大型系统中，依赖关系过紧会导致版本被锁死，过松又易造成版本混乱，`SemVer` 通过一组简单明确的规则约束版本号的配置和增长来应对这一问题。

其核心格式为“主版本号.次版本号.修订号”（`X.Y.Z`），均须为非负整数且不得补零。

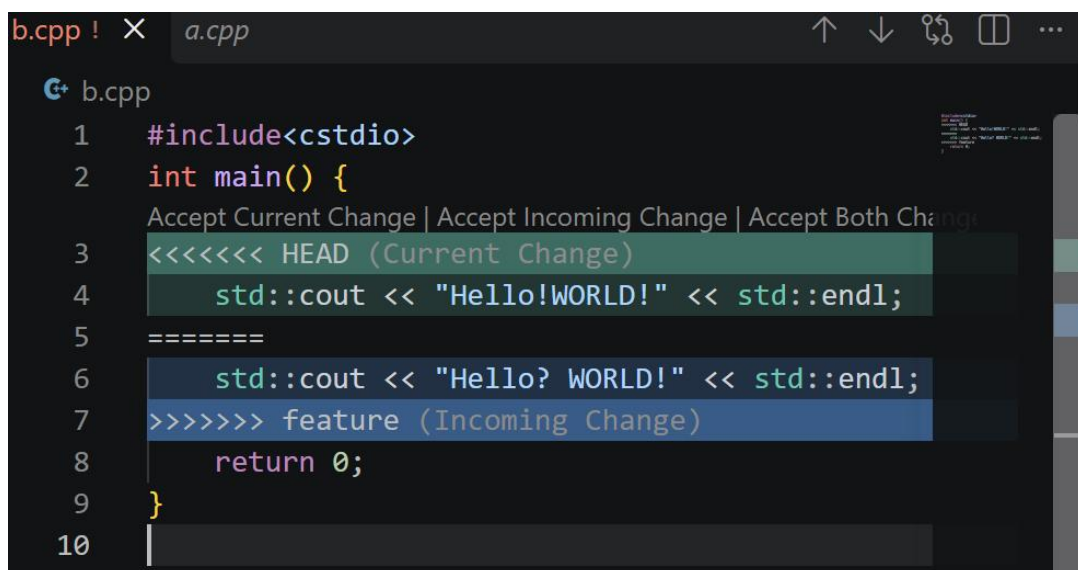
递增规则为：做了不兼容的 `API` 修改时递增主版本号；做了向下兼容的功能性新增时递增次版本号；仅做向下兼容的问题修正时递增修订号。主版本号为零时表示开发初始阶段，`API` 不稳定；`1.0.0` 用于界定公共 `API` 的形成。此外还允许在修订号后追加先行版本号和版本编译信息，并规定了严格的优先级比较规则。

之所以要使用语义化版本控制，是因为版本号本身能向使用者传达相邻版本间的兼容性信息，从而让依赖管理变得可靠而无需频繁推出新的依赖包。它并非全新想法，但只有形成正式规范并坚持遵循，才能真正发挥作用。作为负责任的开发者，应确保每次升级都与版本

号表述一致，并在文档中清晰定义公共 API，让使用者能够依据版本号判断升级的风险与收益。

4.

```
circlewww@LAPTOP-GEBLF004:~/test-git$ git branch feature
circlewww@LAPTOP-GEBLF004:~/test-git$ git branch
feature
* main
circlewww@LAPTOP-GEBLF004:~/test-git$ git switch feature
Switched to branch 'feature'
circlewww@LAPTOP-GEBLF004:~/test-git$ git switch main
M      b.cpp
Switched to branch 'main'
circlewww@LAPTOP-GEBLF004:~/test-git$ git switch feature
Switched to branch 'feature'
circlewww@LAPTOP-GEBLF004:~/test-git$ git switch main
Switched to branch 'main'
circlewww@LAPTOP-GEBLF004:~/test-git$ git merge feature
Auto-merging b.cpp
CONFLICT (content): Merge conflict in b.cpp
Automatic merge failed; fix conflicts and then commit the result.
```



```
b.cpp ! X a.cpp
b.cpp
1  #include<cstdio>
2  int main() {
3  <<<<<< HEAD (Current Change)
4      std::cout << "Hello!WORLD!" << std::endl;
5  =====
6      std::cout << "Hello? WORLD!" << std::endl;
7  >>>>>> feature (Incoming Change)
8      return 0;
9  }
10 |
```



```
b.cpp ! ● a.cpp
b.cpp
1  #include<cstdio>
2  int main() {
3      std::cout << "Hello!WORLD!" << std::endl;
4      std::cout << "Hello? WORLD!" << std::endl;
5      return 0;
6  }
7  
```